

INFORME

Grupo 10

INTEGRANTES:

- Biloni de Bianchetti Camila Alejandra
- Caseres Lautaro
- Fernando Antonio Daniel Lopez
- Gonzalez Avril Araceli
- Ruben Yunes Ramiro

Introducción

Como objetivo principal de este proyecto, nos propusimos modelar un sistema de gestión y control de tráfico urbano valiéndonos de los conocimientos adquiridos.

Se dispuso que el código estuviera dividido en 6 funciones principales, las cuales responden a los requerimientos exigidos

A continuación, enunciamos y explicamos cada función

Transición

Buscamos modelar el cambio de estado del semáforo, optamos por desarrollar una función que reciba como argumento dos parámetros, el color actualmente encendido y el color al cual debe cambiar, como salida obtendremos una lista con el color actual y la orden con el color al que debe mutar, sin embargo, en caso de recibir un color no válido, la función retorna un mensaje con el elemento acción por defecto.

Está basada en un condicional COND, donde en cada rama se comparan los argumentos recibidos con los esperados valiéndonos de la funciones AND y EQ, si hay coincidencia devolverá una lista usando la función LIST con los elementos indicados.

Temporizador

En esta función, buscamos modelar un cronómetro a un un actuador que controla los cambios de colores.

Recibe como argumento un número entero que representa un tiempo en segundos, pensado como tiempo Unix, y devuelve el color que corresponde dentro del ciclo de un semáforo. El ciclo completo dura 216 segundos, distribuidos en tres intervalos:

Rojo durante los primeros 90 segundos, amarillo durante los siguientes 6 segundos y verde durante los últimos 120 segundos. Para lograr que el temporizador sea cíclico, aplicamos recursividad de modo que si el tiempo recibido es mayor o igual a 216, se le resta 216 y se vuelve a llamar a sí misma, repitiendo este proceso hasta que el tiempo quede dentro del rango de un ciclo. Una vez reducido, el condicional cond determina en qué rango del tiempo está ubicado en ese lapso y devuelve un string que indica el color correspondiente a ese momento. De esta manera, la función es pura porque no imprime ni modifica datos externos, su salida es siempre un string con el color correspondiente, y la recursividad asegura que cualquier tiempo entero se pueda mapear correctamente a un estado del semáforo dentro de su ciclo infinito.

Registrar-cambios

En esta instancia se nos propuso desarrollar un pequeño sistema de auditoría, que apoyándose de tres argumentos, devolverá un mensaje por consola el que le serviría al equipo forense, en caso de ser necesario, como historial del comportamiento del semáforo

La función registrar-cambio recibe tres argumentos: un número entero que representa el tiempo en formato epoch, el color anterior y el color nuevo del semáforo. En primera instancia verifica si el tiempo es inválido, es decir menor o igual a cero, o si los colores son iguales: en cualquiera de esos casos devuelve el átomo ERROR como señal de que no se puede registrar el cambio.

En el caso de que las condiciones sean correctas, utiliza la función format para imprimir en la terminal un mensaje que sigue el formato: "Tiempo <epoch>: la luz ha cambiado de <color-anterior> a <color-nuevo>".

Con la intención de mejorar la legibilidad, decidimos cambiar los strings de salida a minúsculas con la función string-downcase. La salida visible de la función es el mensaje en consola, esta función es impura porque genera un efecto externo al imprimir.

Análisis de ciclos de semáforo

Nuestro objetivo principal en esta instancia fue el de calcular la duración total de un ciclo completo del semáforo y verificar si está en los intervalos que la psicología nos recomienda.

Para cumplir con estos requisitos, nos valimos de dos funciones, duracion-ciclo y recomendación-ciclo. La primera recibirá como argumento las duraciones de actividad correspondientes a cada color y devolverá el tiempo total del ciclo, la misma funciona implementando la función reduce. En cuanto a la función Recomendacion-ciclo es simplemente un condicional que luego de recibir la duración total del ciclo, anteriormente calculada con duracion-ciclo, compara si se encuentra dentro de los intervalos recomendados, y de no ser así, devuelve un string con un mensaje que indica si se recomienda aumentar la duración total o disminuirla, o dado el caso, nos avisa que el tiempo requerido para ese ciclo es el óptimo.

ciclos-por-tiempo

Esta función nace desde el planteamiento de que, recibiendo como argumento una cierta cantidad de tiempo expresada en minutos, queremos conocer la cantidad de ciclos que cumple el equipo en dicho lapso. Para eso desarrollamos ciclos-por-tiempo, una función que trabaja con variables locales haciendo uso de la funcionalidad let*, dentro de esta calcula el valor de dos variables, una expresará el valor recibido como argumento en unidades de segundos y la otra el valor duración total de un ciclo, ya con estos valores adquiridos, usamos la función Floor argumentada por las dos variables anteriormente mencionadas, y para evitar los valores secundarios que Floor produce, como por ejemplo el resto, se utiliza values

Distribución Temporal

La función `distribucion-temporal` procesa los datos de manera ordenada para calcular qué porcentaje del ciclo completo ocupa cada color del semáforo. El argumento de entrada es tiempos, una lista con tres elementos que representan la duración en segundos de cada color dentro de un ciclo: por ejemplo (90 6 120) para rojo, amarillo y verde respectivamente. Lo primero que hace la función es aplicar `mapcar`, que recorre en paralelo la lista de tiempos y la lista de etiquetas de colores ("rojo" "amarillo" "verde"). En cada iteración, la función anónima lambda recibe dos valores: x, que es la duración en segundos de un color, y, que es el nombre del color correspondiente. Luego verifica con `numberp` que x sea un número válido. Si lo es, construye una lista con dos elementos: el nombre del color y el porcentaje que ese tiempo representa respecto a la duración total del ciclo. Para calcular ese porcentaje, divide x por el resultado de la función `duracion-ciclo` aplicada a la lista completa de tiempos, y multiplica por 100 para expresarlo en porcentaje. El resultado final es una lista de listas, donde cada sublista contiene el nombre del color y su porcentaje de participación en el ciclo

Fase 2

En este punto, se nos propuso integrar librerías al código actual, con el objetivo de mejorar las salidas, puesto que imprimir el resultado en formato epoch no lo consideramos muy cómodo ni legible

Como primer paso Instalamos el gestor de paquetes Quicksip que nos permite cargar librerías como Local Time en Lisp. Utilizamos la función `unixToTime` para convertir números epoch a objetos `TimeStamp` y formatearlos como fechas legibles. Al final añadimos una condición para que solo se registre un cambio cuando el color realmente sea diferente, y así quedó todo funcionando de forma clara. Para una mayor descripción de este punto, se sugiere visitar la bitácora de depuración, anexada en el mismo repositorio, ya que la misma presentó más inconvenientes de los presentados en esta redacción.

Estudio comparativo

OCaml (Objective Caml) es el principal lenguaje de la familia ML (Meta Language). Es un lenguaje de programación imperativo y funcional de propósito general que está orientado a objetos. Se caracteriza por su sistema de tipos estático con inferencia de tipos, lo que permite detectar errores en tiempo de compilación sin necesidad de declarar explícitamente los tipos de las variables y funciones.

Fue escrito en 1996 por Xavier Leroy y Jérôme Vouillon, Damien Doligez y Didier Rémy en el INRIA de Francia

Empresas como Meta, Docker y Bloomberg L.P. utilizan OCaml en diferentes ámbitos (por ejemplo, Meta utilizó OCaml para la creación de herramientas como Flow, Hack, Infer y entre otros más)

En OCaml, la inferencia de tipos asigna automáticamente un tipo de datos a una función sin necesidad de que el programador lo escriba, por lo que no tuvimos la necesidad de declarar explícitamente el tipo porque el compilador sigue verificando estrictamente los tipos. Si se intenta utilizar un argumento de un tipo incompatible, el programa no compila.

En Lisp, las estructuras de control como (cond ...) o (if ...) evalúan condiciones lógicas de verdadero/falso. Al ser OCaml un lenguaje "orientado a expresiones", significa que absolutamente todo (incluyendo los condicionales y los bloques match) devuelve un valor. La estructura "match...with" no funciona evaluando "verdadero o falso", sino haciendo

Pattern Matching (Pattern Matching es una técnica que permite comparar un dato contra distintos patrones posibles y ejecutar una acción según el patrón que coincida.) sobre la estructura del dato. En OCaml se implementa mediante la estructura “match ... with”, que facilita trabajar con tipos de datos definidos por el usuario y evita el uso de múltiples condicionales anidados. Además, hace que el código sea más legible y fácil de mantener.

Podemos resaltar que la experiencia de estudiar e implementar la lógica del semáforo en OCaml nos permitió contrastar las diferencias entre un lenguaje de tipado dinámico como Lisp y uno estático como OCaml. A través del desarrollo de la función “transición”, comprobamos cómo la creación de tipos de datos propios como “estado” combinada con la técnica de Pattern Matching ofrece una estructura mucho más robusta y legible que los condicionales de Lisp, obligándonos a cubrir de forma explícita y segura cada escenario posible. Por otra parte, la implementación del “temporizador”, mediante una estrategia recursiva pura, nos demostró cómo el paradigma funcional resulta efectivo para resolver problemas matemáticos continuos manteniendo la naturaleza pura y no destructiva de las funciones. En definitiva, confirmamos que la inferencia de tipos de OCaml elimina la rigidez de tener que declarar datos manualmente y tiene la gran ventaja de que encuentra los errores al compilar, mucho antes de que el código se ponga a ejecutar.

iteración 2: Intermittencia de Seguridad

A partir de este punto, ya se han modelado las funciones principales, lo siguientes es una adaptación de las funciones desarrolladas hasta ahora para poder incluir la intermitencia de seguridad, la cual consiste en presentar un parpadeo en la luz actual antes del cambio de estado. Las únicas funciones que debieron ser modificadas para cumplir con el objetivo fueron las siguientes dos: **transición** , **temporizador**, ambas con un simple cambio en donde se incluyen los casos de intermitencia aumentando la cantidad de proposiciones en el condicional, pero ambas mantienen su estructura básica.

Validación de datos

Como medida de seguridad hemos optado por un conjunto de funciones dedicadas a la validación de entrada de datos, las cuales serán invocadas en un menú junto a las funciones principales, seguidamente las anunciaremos acompañadas de una breve explicación de su objetivo:

validar-estados: Verifica que el valor ingresado sea uno de los estados básicos del semáforo (rojo, amarillo, verde). Si no lo es, vuelve a pedir la entrada hasta que sea correcta.

validar-estados2: Similar a la anterior, pero valida estados con el prefijo en- (en-rojo, en-amarillo, en-verde). Repite la solicitud hasta recibir un valor válido.

validar-número :Comprueba que el dato ingresado sea un número entero positivo. Si no cumple la condición, solicita nuevamente la entrada.

validar-lista :Exige que el usuario ingrese una lista de exactamente 3 números enteros positivos. Si no se cumple, muestra un mensaje de error y vuelve a pedir la lista.

validar-opción:Controla que la opción ingresada sea un número entero entre 0 y 6. Si el valor está fuera de rango, solicita otra vez la entrada.

Persistencia de Datos

En nuestro sistema, la **persistencia de datos** se construye a partir de la interacción entre tres funciones: ejecutar-guardar, informe y el menu, todas ellas unidas por la variable compartida historial, que no es global sino que se transmite como argumento entre las funciones.

ejecutar-guardar: se basa en la función registrar-cambio para decidir si un evento debe agregarse al historial. Si registrar-cambio devuelve una cadena válida, entonces la función lo incorpora al historial mediante cons. Si no, devuelve el historial sin modificaciones. De esta manera, solo los cambios válidos se acumulan.

menu: es la función especial que agregamos para mejorar la experiencia del usuario. Está construida sobre un cond, que evalúa las distintas opciones ingresadas, y aplica recursividad para volver a mostrarse después de cada acción, manteniendo el flujo activo. Al seleccionar la opción **0**, el menu llama a informe, guarda todo el historial en el archivo y finaliza el programa con un mensaje de despedida.

informe: al finalizar, toma el historial acumulado y lo escribe en un archivo externo. Aquí usamos with-open-file, que abre el archivo en modo escritura y garantiza que se cierre automáticamente al terminar, evitando fugas o errores. Dentro de este bloque aplicamos mapcar para recorrer el historial y escribir cada registro en el archivo, asegurando que todos los eventos queden documentados.

Conclusión

En este proceso sentimos que realmente aumentamos nuestro aprendizaje y arraigamos los conocimientos, porque nos permitió explorar funciones y librerías que antes no utilizamos. Fue una experiencia enriquecedora de trabajo en grupo: normalmente estamos acostumbrados a trabajar solos, pero esta dinámica nos ayudó a fortalecer la colaboración y a integrar herramientas que en la vida profesional son de uso cotidiano, como GitHub. Además, el desarrollo por ramas nos dio una forma eficaz de trabajar todos sobre el mismo código sin pisar el trabajo de otro, lo que nos acercó a prácticas reales de equipo y nos preparó mejor para entornos profesionales.

Bitácora: Finalmente, cabe destacar la importancia de la bitácora de depuración elaborada durante el desarrollo. En ella se registraron de manera sistemática los errores encontrados, las hipótesis de causa y las soluciones aplicadas. Este registro no solo permitió corregir fallos oportunamente, sino también consolidar el aprendizaje y dejar evidencia del proceso de mejora continua. En caso de requerir información más detallada sobre las dificultades afrontadas, se puede consultar la bitácora disponible en el mismo repositorio donde se encuentra este informe.

9. Bibliografía

- OCaml
 - <https://ocaml.org/>
 - <https://ocaml.org/play>